

Conduit — Frontend Build Brief & Storyboard

Read this first: This site is the face of Conduit. The backend is already built and genuinely excellent — real engine, real auth, real billing, 400+ tests. The frontend has to match that bar.

We are not looking for "fine" or "it works." We want a frontend that makes a developer stop and think "damn, this is a serious product." Vercel, Linear, Stripe, Railway — that tier. If it looks like a template, it's wrong. This is the standard for the whole build.

What Conduit is (so you build with understanding): Conduit turns a plain-English description into a working MCP server — a connector that lets an AI assistant actually *do* things (call APIs, query databases). The user types what they want, Conduit generates and validates real code, they download it and plug it into Claude, Cursor, or any MCP client. The magic is: *describe it -> get working code*. The whole site should make that magic feel effortless and premium.

THE BAR (non-negotiable design standard)

- **Aesthetic:** dark, sharp, technical, confident. Developer-tool premium. Think Linear's polish, Vercel's cleanliness, Stripe's trust.
- **Typography:** a crisp sans for UI, a good monospace for all code. Type hierarchy deliberate — not everything the same size.
- **Motion:** subtle, fast, purposeful. Micro-interactions on buttons, smooth transitions, a satisfying generate animation. Never janky, never slow.
- **Whitespace:** generous. Cramped = amateur. Let it breathe.
- **Consistency:** one color system, one spacing scale, one component style everywhere. It should feel like one hand made it.
- **Speed:** it must *feel* instant. Loading states everywhere, nothing that hangs on a white screen.
- **The Generate page is the star.** Pour the most craft here. When a user types a description and watches a real connector appear, that moment should feel like magic. That single interaction sells the product.

If any screen looks like an unstyled Bootstrap form, it fails the bar. Every screen should look intentional.

TECH NOTES

- **Auth:** Supabase Auth (email/password + magic link, optional Google/GitHub). Logged-in user carries a Supabase session JWT.
 - **Calling the backend:** send the JWT as `Authorization: Bearer <jwt>` on every dashboard API call.
 - **API base URL:** from an env var (e.g. `VITE_API_URL`) — never hardcoded.
 - **Two credential types the backend uses:** the Supabase JWT (dashboard actions) and `cndt_` API keys (which users generate to call Conduit from their own code).
 - **Suggested stack:** Next.js or Vite + React + Tailwind. (Builder's choice, but modern React.)
 - **CORS** is already handled backend-side.
 - **The backend is the source of truth** for exact request/response shapes. Endpoints below are real and tested — confirm exact fields against the API handlers.
-

PART 1 — PUBLIC SITE (logged-out)

PAGE 1 — Landing /

This page has one job: make a developer believe Conduit is real and worth signing up for. It should look like a product that already has thousands of users.

Nav (sticky, subtle blur on scroll):

- Left: Conduit wordmark
- Right: `Docs · Sign in · Get started` (primary, stands out)

Hero (big, confident):

- **Headline:** "Describe it. Get a working MCP server."
- **Sub:** "Conduit turns plain English into a real, validated MCP connector your AI can use — no boilerplate, no guesswork."
- **Primary CTA:** `Get started free -> /signup`
- **Secondary:** `See it work -> scrolls to demo`
- **Hero visual (make this great):** an animated/high-fidelity mockup showing the transformation — a line of English on the left ("look up an order by id in Postgres") flowing into clean generated code on the right. This visual IS the pitch. Spend time here.

"How it works" — 3 steps, clean horizontal cards:

1. **Describe** — type what you want in plain English
2. **Generate** — Conduit builds and validates a complete MCP server
3. **Connect** — download and plug into Claude, Cursor, any MCP client

Interactive demo teaser (strong if you can):

- A text box with 3 preset buttons: [Stripe API](#) · [Postgres query](#) · [Slack message](#). Clicking one shows a beautifully formatted sample of the connector Conduit would generate. Makes the value tangible before signup. Use pre-baked examples so it's instant and needs no auth.

What it supports — feature grid, iconed:

- REST APIs (with auth) · Postgres · MySQL · GraphQL · static data
- "Every connector is validated against the MCP spec"
- "Honest — if it can't build something, it tells you instead of faking it"

Pricing — 2 clean cards:

- **Free** — 200 connectors/month · all backing types · \$0 -> [Start free](#)
- **Pro** — 10,000 connectors/month · priority · [\\$\[PRICE — Abdullah to set\]/mo](#) -> [Upgrade](#)
- Make the Pro card the visually "recommended" one (subtle highlight/border).

Footer: Docs · Pricing · Sign in · (later: Terms, Privacy) · "Built by Poulta Ventures"

Backend: none needed (static). The demo teaser uses pre-baked examples, no API call.

PAGE 2 — Sign Up </signup>

Clean, centered, minimal — but premium, not a boring form. Nice card, good spacing, maybe a subtle branded side panel.

- Email · Password (or magic-link option) · [Create account](#)
- Optional: Google / GitHub buttons
- "Already have an account? Sign in" -> </login>
- **Backend:** Supabase Auth [signUp](#). On success -> session created -> redirect to </dashboard>. (Backend auto-creates the Conduit account on first authenticated call — already built.)

PAGE 3 — Sign In </login>

Same premium treatment as signup.

- Email · Password (or magic link) · [Sign in](#)
 - "Forgot password?" -> Supabase reset
 - "No account? Sign up" -> </signup>
 - **Backend:** Supabase Auth [signInWithPassword](#) -> redirect to </dashboard>.
-

PART 2 — DASHBOARD (logged-in)

Every page here needs a valid session. No session -> redirect to </login>. Every API call sends the JWT.

Dashboard shell (shared on all logged-in pages):

- **Sidebar (left, clean, iconed):** Conduit logo · [Generate](#) · [My Connectors](#) · [API Keys](#) · [Usage](#) · [Billing](#) · (pinned bottom) account email + [Sign out](#)
 - **Main area:** the active page
 - Sidebar highlights the active page. Collapsible on mobile.
-

PAGE 4 — Generate </dashboard> * THE STAR — make this exceptional

This is the heart of the product. Spend your best effort here. The experience should feel like magic: type -> watch it build -> get real code.

Layout:

1. **Prompt area (front and center):** a large, inviting text box. Placeholder: *"Describe the connector you want — e.g. 'Look up an order by id from our Postgres orders table' or 'Send a Slack message to a channel'"*
2. **[Generate connector](#) button** — primary, satisfying to click.
3. **Generating state (make it feel alive):** a smooth animated sequence — "Understanding your request..." -> "Building the connector..." -> "Validating..." — not just a dead spinner. This moment should feel like watching something get built.
4. **Result — SUCCESS:**

- **Spec summary** at top: connector name, each tool with a one-line description (clean cards).
 - **Generated files** in tabs: `server.ts` · `package.json` · `README.md` — syntax-highlighted, monospace, dark code theme.
 - **A green Validated badge** — reassures it's real and correct.
 - **Download button** — zips the files.
 - **Copy button** per file.
 - A short "How to use this" hint: a one-liner on plugging it into Claude/Cursor.
5. **Result — REFUSAL:** if Conduit honestly can't build it (e.g. "pure math, no API needed" / "too vague"), show it *gracefully and helpfully*, not like an error: "Conduit couldn't build this one: [reason]. Try describing an API call or a database query." This honesty is a feature — present it calm and clear.
 6. **Result — QUOTA HIT:** if they're out of monthly generations -> a clean upgrade nudge: "You've used all 200 connectors this month. Upgrade to Pro for 10,000." + **Upgrade** -> `/dashboard/billing`.

Backend:

- **POST** `/v1/connectors/generate` — body { `description` }, JWT header. Returns spec + files, OR a refusal, OR 402 quota-exceeded. Match exact shape to the backend.

Buttons: **Generate** (calls endpoint) · **Download** (zips files client-side) · **Copy** per file · **Upgrade** (quota case only).

PAGE 5 — My Connectors `/dashboard/connectors`

The user's saved history of everything generated. Organized and browsable, not a dull table.

- **List:** each item = connector name, description, date, number of tools. Nice cards or a well-styled table.
 - **Click an item** -> detail view: the spec + files again, with **Download** and **Copy**.
 - **Empty state (inviting, not blank):** "No connectors yet — generate your first one." + button -> `/dashboard`.
 - **Backend:** **GET** `/v1/connectors/list` (JWT) -> the user's connectors. **GET** `/v1/connectors/get?id=...` -> one connector's detail. (Backend enforces users only see their own — already built.)
-

PAGE 6 — API Keys </dashboard/keys>

Where users create the `cnidt_` keys they'll use to call Conduit from their own code.

- **Key list:** each row = key name, prefix only (e.g. `cnidt_abc...` — never the full key), created date, last used, [Revoke](#).
 - **Generate new key** -> small modal: "Name this key" + [Create](#).
 - **On creation — critical UX:** show the FULL key ONCE in a highlighted box with a clear warning: "**Copy this now — you won't be able to see it again.**" + a [Copy](#) button. (Backend stores only a hash; the raw key exists once. Handle the one-time reveal well.)
 - **Revoke** -> confirm dialog ("Revoke this key? Anything using it will stop working.") -> then revoke.
 - **Backend:** `GET /v1/keys/list` · `POST /v1/keys/generate` (body { `name` }, returns raw key once) · `POST /v1/keys/revoke` (body { `id` }). All JWT.
-

PAGE 7 — Usage </dashboard/usage>

Clean, at-a-glance monthly quota view.

- **Big central number:** "X / 200 connectors this month" (or / 10,000 on Pro).
 - **A nice progress bar** (fills, shifts color as it nears the limit).
 - **Plan badge:** Free / Pro.
 - **Reset date:** "resets on the 1st."
 - **Near/at limit** -> an [Upgrade](#) nudge -> </dashboard/billing>.
 - **Backend:** `GET /v1/usage/summary` (JWT) -> usage count + plan limit.
-

PAGE 8 — Billing </dashboard/billing>

Plan management and upgrade. Must feel trustworthy — people pay here.

- **Current plan card:** Free or Pro, the limit, the status.
- **On Free:** an attractive "Upgrade to Pro" card — benefits (10,000/mo), price, [Upgrade to Pro](#) button.
- **On Pro:** show the active subscription + [Manage subscription](#) (Stripe customer portal, or a cancel flow).
- **Upgrade flow:** [Upgrade to Pro](#) -> calls checkout -> **redirect the browser to the Stripe checkout URL** it returns. Stripe handles payment, then redirects back to a

success URL you set (e.g. `/dashboard/billing?success=true`). On return with `?success=true`, show a success toast and refresh the plan.

- **Backend:** `GET /v1/billing/plan` (JWT) -> plan, limit, status, plan catalogue. `POST /v1/billing/checkout` (JWT) -> Stripe checkout URL. (The upgrade is finalized by a backend webhook — the frontend just starts checkout and re-reads the plan after.)

PAGE 9 — Settings `/dashboard/settings` (simple)

- Show email · `Change password` (Supabase) · `Sign out` · (later: delete account).
Clean and minimal.

PART 3 — THE CONNECTION MAP (every action -> backend endpoint)

Action	Page	Endpoint	Auth
Sign up	/signup	Supabase Auth	—

Action	Page	Endpoint	Auth
Sign in	/login	Supabase Auth	—
Generate a connector	/dashboard	POST /v1/connectors/generate	JWT
List my connectors	/connectors	GET /v1/connectors/list	JWT
View one connector	/connectors	GET /v1/connectors/get	JWT
List API keys	/keys	GET /v1/keys/list	JWT
Create API key	/keys	POST /v1/keys/generate	JWT
Revoke API key	/keys	POST /v1/keys/revoke	JWT
See usage	/usage	GET /v1/usage/summary	JWT
See plan	/billing	GET /v1/billing/plan	JWT
Start upgrade	/billing	POST /v1/billing/checkout	JWT

Confirm exact request/response fields against the backend API handlers — the endpoints are real and tested; the backend is the source of truth.

PART 4 — STATES TO HANDLE ON EVERY PAGE (the polish that separates pro from amateur)

- **Loading** — skeletons/spinners on every API call, never a dead white screen
- **Empty** — inviting empty states (no connectors, no keys), never just blank
- **Error** — a friendly message + retry, never a raw error dump
- **Quota exceeded (402)** — the upgrade nudge on generate
- **Refusal** — Conduit's honest "can't build this" message, shown gracefully
- **Session expired** — redirect to /login cleanly
- **One-time key reveal** — the API key shown once, copy + warning, handled carefully

Handling these well is a large part of what makes the site feel *real*. Don't skip them.

PART 5 — DESIGN DIRECTION (summary)

- Dark, premium, developer-grade. Linear / Vercel / Stripe tier.
- Deliberate typography, generous whitespace, one consistent system.
- Subtle, fast motion. A great generate animation.
- Monospace + syntax highlighting for all code.
- **The Generate page is the star — make that moment feel like magic.**
- Fully responsive (desktop-first, but must work on tablet/mobile).

Bottom line: this should look like a product people already trust with real work. Nothing templated, nothing half-finished. Make it genuinely great.

PART 6 — MOTION, TRANSITIONS & EFFECTS (bring it to life)

A static site, however clean, feels dead. Conduit should feel *alive* and premium. Put real craft into motion — it's a big part of what makes it feel high-end.

What we want:

- **The generate moment** — when someone hits Generate, don't just pop the result in. Animate it: the "Understanding -> Building -> Validating" steps should transition smoothly, and the connector result should reveal with a nice fade/slide-up. This is the signature moment — make it satisfying.
- **The hero** — the "describe -> code" visual on the landing page should animate: the English text types or fades in, then the generated code appears beside it. Make a visitor *feel* the magic in the first 3 seconds.
- **Buttons & cards** — subtle hover states everywhere: gentle lift, a soft glow on the accent, smooth color transitions. Nothing static.
- **Page transitions** — smooth fades between pages/routes, not hard cuts.
- **Loading** — skeleton shimmers, not blank screens or ugly spinners.
- **Micro-interactions** — the "Copy" button gives a little "Copied!" confirmation; the validated badge can subtly pop in; number counters can count up.
- **Scroll reveals** on the landing page — sections fade/slide in gently as you scroll.

The rule: motion should be *fast, smooth, and purposeful* — never slow, never gratuitous, never janky. Think Linear and Vercel: you barely notice the animations, but the whole thing feels effortless and expensive. If a transition feels laggy or distracting, cut it.

Do NOT overdo it into something flashy or gimmicky. Tasteful and smooth beats loud and busy every time. But the site should never feel flat or static — it should feel like it's breathing.